CS-303 Assignment 2

(25 points)

Q1. (15 points) Create a generic singly linked list class in Java named SingleLinkedList<T>. Your class must maintain the following instance variables:

- head (reference to the first node)
- tail (reference to the last node)
- size (an int tracking the number of elements)

Your list should support the following methods:

- void addFirst(T item)
- void addLast(T item)
- T removeFirst()
- T removeLast()
- T getFirst()
- T getLast()
- boolean isEmpty()

Index-based operations:

- void insert(int index, T item): Insert item at position index (starting at 0). If index is greater than the current size, insert at the end.
- boolean remove(int index): Remove the item at position index. Return true if successful; return false if index is out of bounds.
- int find(T item): Return the index of the first occurrence of item if found; otherwise return the size of the list.

Notes / Expectations

Implement an internal private static class Node<T> containing:

- T data
- Node<T> next

Your methods must correctly update head, tail, and size and for removeFirst()/ removeLast() / getFirst() / getLast():

If the list is empty, you may either throw NoSuchElementException, or

return null (choose one approach and be consistent—document it in your README).

Q2 (10 points) Write a Java program that implements a stack of integers using an ArrayList<Integer> as the backing store.

Your stack class must provide at least the following public methods:

- void push(int value) — push an element onto the top of the stack
- int pop() — remove and return the top element
- int peek() — return the top element without removing it
- boolean isEmpty() — return true if the stack has no elements
- int size() — return the number of elements in the stack *(Java-style standard)*
- double average() — return the average of all elements currently in the stack *(custom method)*

In your main() method, perform the following:

1. Instantiate a stack object.
2. Call isEmpty() and display the result.
3. Call push(...) several times to add integer values.
4. Call pop() once and display the removed value.
5. Call peek() and display the current top value.
6. Call average() and display the average of the stack elements.

Notes / Expectations

- Treat the end of the ArrayList as the top of the stack (LIFO):
  - push → list.add(value)
  - pop/peek → operate on list.get(list.size() - 1)
- Handle error cases:
  - If pop() or peek() is called on an empty stack, throw a Java-appropriate exception such as EmptyStackException (from java.util) or NoSuchElementException

Submission guidelines:

1) All functionality must be implemented using methods and classes.
2) Your code must be well-commented and easy to follow.
3) Create a README.md report that includes:
   a. How to compile and run the program
   b. Brief explanation of your design
   c. Screenshots of sample output (or copy/paste console output)
4) Upload all code and the README to GitHub.
5) Submit the GitHub repository link on Canvas by the due date.